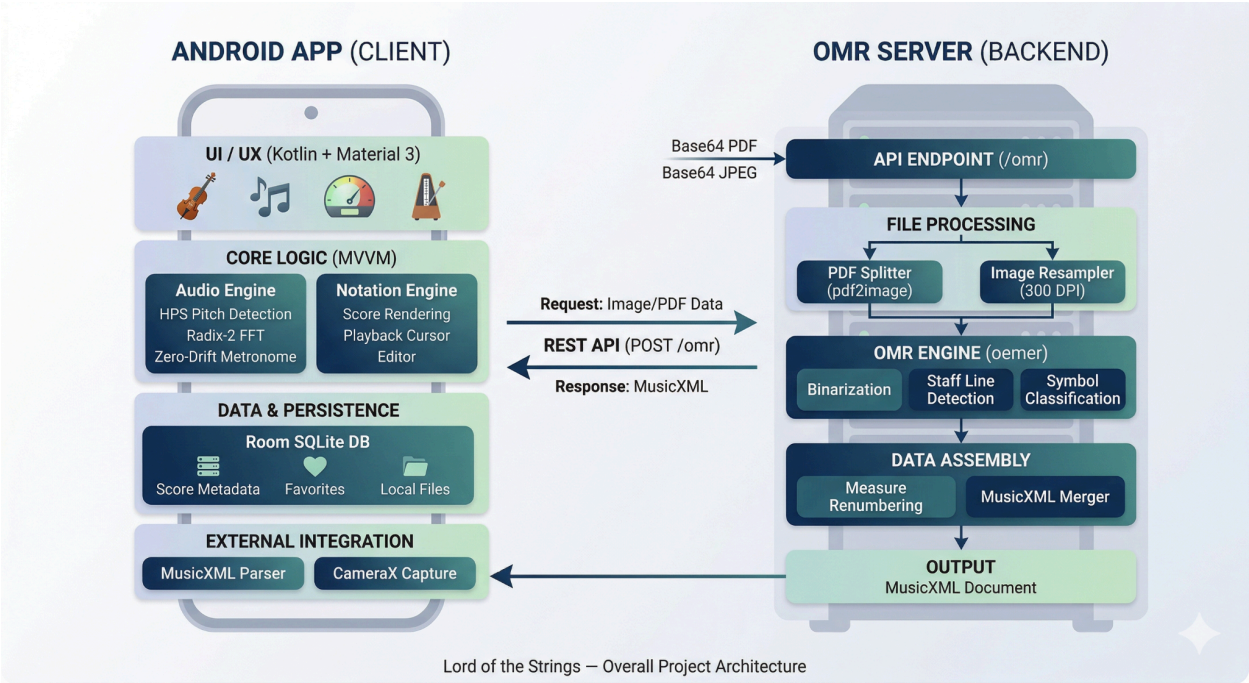
This architecture documentation outlines the technical design of the **Lord of the Strings** system, a mobile platform for cello musicians. The system consists of a native Android application and a specialized Python-based OMR (Optical Music Recognition) server.



Lord of the Strings — Overall Project Architecture

---

1. High-Level System Architecture

The system follows a client-server architecture where the Android app handles real-time audio processing and user interaction, while heavy computational tasks (like image-to-sheet-music conversion) are offloaded to a dedicated server.

- **Android App**: Built with Kotlin, it manages the local SQLite database (Room), real-time pitch detection (HPS), and music notation rendering.
- **OMR Server**: A Flask-based Python environment that uses the `oemer` library to process complex visual data into MusicXML format.
- **Communication**: The app communicates with the server via a REST API (`POST /omr`) using Base64-encoded payloads.

---

2. Android App Architecture (Client)

The mobile application is structured using a **Model-View-ViewModel (MVVM)** pattern to ensure a clean separation of concerns.

Navigation & UI Layer

- **Jetpack Navigation**: A single-activity architecture (`MainActivity`) manages fragments for the Library, Tuner, Metronome, and Score Viewer.
- **Custom Views**: High-performance UI components like `ScoreCanvasView` for notation rendering and `TunerGaugeView` for the needle animation.

Domain & Data Layer

- **Domain Model**: Defines the core musical entities such as `Score`, `Measure`, `Note`, and `Pitch`.
- **Data Layer**: Uses **Room Persistence** to store score metadata, favorite status, and local file references.

Specialized Audio & OMR Engines

- **HPS Pitch Detection**: A custom radix-2 DIF FFT implementation for cello-specific frequency detection.
- **Metronome Engine**: An `AudioTrack` based engine designed for zero-drift timing using sample-count synchronization.
- **Local OMR Fallback**: If the server is unreachable, the app utilizes a local OMR pipeline (binarization and staff detection).

---

3. OMR Server Architecture (Backend)

The server is designed for high-latency, high-accuracy processing of musical scores.

Processing Pipeline:

1. **API Entry**: Receives a Base64 PDF or image via the `/omr` endpoint.
2. **PDF Conversion**: If the input is a PDF, `pdf2image` (Poppler) renders pages into 300 DPI PNG images.
3. **Symbol Recognition**: The `oemer` engine performs staff line detection and connected-component classification.
4. **XML Assembly**: Individual page results are merged. The server logic iterates through `<measure>` elements, renumbering them into a single coherent MusicXML document.
5. **Response**: The final MusicXML string is returned to the Android client.

---

4. Data Flow: Score Import

The following sequence describes how a physical score becomes a digital, playable entity:

- **Capture**: The user takes a photo using the `CameraX` integration.
- **Transmission**: The app sends the image to the configured OMR Server URL (e.g., `http://10.0.2.2:5000` for emulators).
- **Transformation**: The server processes the image (taking 30s to 3m) and returns MusicXML.

- **Ingestion**: The app's `MusicXmlParser` converts the XML into the internal `Score` model, which is then saved to the Room database.
- **Visualization**: The `ScoreViewerFragment` renders the notation on the zoomable canvas for playback and editing.